

CBAM calculator: what shipped in August, and what it still cannot do

Internal brief · 8 August 2026 · prepared for the founding team

This covers one branch of work on the public CBAM calculator: multi-line estimates, a real annual threshold verdict, and CSV/print export. It does not restate the formulas, the production-route matrix or the routing logic behind any of this — those are in `cbam-engine-reference.pdf`, and this brief points to it rather than repeat it. Every figure below was checked against the live pack and the test suite on 8 August 2026, run today rather than carried forward from an earlier draft.

What shipped

Four changes to the public calculator, one branch, nothing in the vendored engine touched — `node scripts/cbam-sync-check.mjs` still reports all eleven engine files byte-identical to the committed manifest.

Multi-line estimates. The calculator priced one line at a time. It now takes a whole bill of materials: add a line and it prices through the same engine as before, with nothing new invented; remove one; a line the engine cannot price renders its own refusal card, naming what is missing, and is excluded from the total rather than blanking the estimate for every other line on the sheet.

A real annual de minimis verdict. The threshold in Reg (EU) 2023/956 Art 2(3), as amended by Reg (EU) 2025/2083, is 50 t per importer per calendar year, across cement, aluminium, fertilisers and iron & steel. One line can prove an importer is *above* that — but never *below* it, since below requires knowing the whole year — so the calculator used to return `indeterminate` for every importer under the threshold, which is most small importers. It now groups every line by calendar year and, only when the user explicitly ticks "these are all my 2026 imports," can return `below_threshold` for that year. A year that is provably above 50 t reports `above_threshold` regardless of the tick: that is provable from partial data, and a checkbox must not be allowed to gate a fact.

An attestation the tool never assumes. The tick starts unticked, for every year, every time. It is dropped automatically the moment the set of lines belonging to that year changes at all — by adding a line into the year or removing one out of it — because a tick is a statement about one specific list, and any edit produces a different list nobody has attested to. Every surface that shows a below-threshold verdict says on its face that it rests on the user's own statement, unverified by the Commission or by us.

Export: a CSV and a printable audit document. The CSV carries one row per line, every engine figure at full precision — no rounding, no locale formatting — with the legal locator that authorises it in the adjacent column: a benchmark's Annex line, the CBAM factor's Directive article, the CSCF's status. The printable document is built as a print stylesheet through the browser's own `window.print()` — no PDF library — in

four sections: what was asked, what was computed, on what authority, and, in its fourth section, what none of the above can tell you. That fourth section is substantially the same list as this brief's next section, printed onto every export a user takes away.

Why it matters commercially

The 3 August competitive brief on kolum (`docs/kolum-competitive-brief.md`) named multi-line input as "the single largest usability gap" in our product and the cheapest one to close. That recommendation is done, and the brief has been updated to say so. The same brief called our CSCF stance the sharper contrast with kolum's single confident number, but judged the honesty read as a refusal rather than a reason to trust us — its recommendation 4. The floor-not-ceiling correction and the printed document's fourth section are what close most of that gap; see the updated brief for the full account of what still isn't closed.

Both changes matter beyond feature parity. Before this branch, every importer whose whole year sat under the 50 t threshold got the same non-answer as one deep over it: `indeterminate` . That is the single most common real question a small importer has — does CBAM apply to me at all — and the tool could not say yes or no to anyone. It still will not say so unprompted; it now says so conditioned on the user's own statement, which is the honest version of the answer this tool exists to give.

The export turns an estimate from a number on a screen that resets on refresh into something a consultancy client or an importer's own compliance desk can keep, forward and file. That fits the calculator's actual job — a credibility instrument, not a lead funnel — better than a bare on-screen figure did.

What is not done: the sourcing comparison — kolum's highest-value feature, and the one our own rule pack is already positioned to build — remains unbuilt. On the updated brief's own re-ranking, it is now the top open recommendation.

What it still cannot do

The scenario assumes CSCF = 1 — the maximum the factor can legally be — so every figure the calculator shows is a floor, not a midpoint. The real bill cannot be lower than what is shown, and may be higher, once the Commission publishes the true factor. The cross-sectoral correction factor is unpublished for 2026–2030, and 94.3% of every answer the calculator produces today is this exact labelled scenario rather than a settled figure — measured directly against the live pack: 2,656 of 2,816 (good, origin, route) combinations swept across the catalogue land on `cscf_pending` ; none land on a final `ok` .

That direction matters, and it was stated backwards in the shipped product until this branch. The on-screen wording for this scenario used to read: *"the last value actually set (2021–25). The real figure cannot be higher, and may be lower."* That is the opposite of what is true — CSCF only ever reduces the free-allocation deduction that offsets a bill, so pinning it at its legal maximum computes the largest deduction the law can

produce, which is the smallest bill the good can owe. A reader who trusted the old wording and provisioned to the number shown would have under-provisioned for a bill that, once the real factor lands, can only hold or rise. Fixed 8 August 2026: the card now reads *"...the largest the factor can legally be... this is a floor: the real figure cannot be lower, and may be higher."* Since 94.3% of all answers are this scenario, the direction of that one sentence governs almost everything the tool says.

Import dates in 2027 or later fail closed, on the whole line, not just the price. Certificate prices are published one quarter at a time; the pack carries four rows, all 2026 (Q1 €75.36 and Q2 €75.28 published; Q3 and Q4 not yet). A 2026 Q3 or Q4 import still prices normally — a certificate's figure appears with no euro cost attached, because the price row exists and is simply marked pending. A 2027 import gets nothing: no certificates, no cost, because no price row exists for that year at all, and price resolution runs unconditionally before the calculator can decide whether to show a what-if or a settled figure. The resulting card looks exactly like a missing-benchmark refusal — same "No estimate" tag, same generic reason sentence — and only the coded selector printed beneath it (`certificate-price/2027-Q1` , for instance) names the actual gap. A user who does not read that far sees an unexplained refusal, not a message that says certificate prices past 2026 are not published yet.

19 goods with combined production routes are unresolved. IR (EU) 2025/2621 publishes some goods against a combined code — `(C)/(F)` or `(E)/(H)` — where the process is known but the steel grade is not; the benchmark table always publishes exactly one of the two halves, so the fix is a straightforward split-and-match. The resolver instead requires an exact string match, so a combined code matches nothing and those 19 goods fail closed. Backed by both regulations; not yet implemented.

Article 9 deductions for carbon already priced at origin are not modelled. CBAM lets an importer deduct a carbon price already paid in the country of origin; the implementing act for that mechanism is still a draft, so the calculator does not apply it. Every figure the tool shows is conservative by this omission alone — an importer who genuinely paid a carbon price at origin owes less than what is shown, never more.

The 41,100 default emission factors have never been reconciled against the Commission's own Annex I. The engine reference calls this the largest unaudited surface in the system, and that stands. The Annex runs to 2,400 pages; nothing has been found wrong in it, and nothing has been checked exhaustively either.

Column A — the actual-process-data path — is deliberately not exposed. It is implemented and audited, and correct only against a verified process-only figure with a declared precursor list. A screening tool fed a CN code and a country has neither, and offering the control would let a wrong pairing move the bill against the user with no warning. See §2.3 of the engine reference for the full reasoning. This is a considered decision, not a gap — recommendation 3 of the kolum brief holds it as a standing hold, unchanged by this branch.

How it was verified

`npm run verify` — `astro check` (181 files, 0 errors, 0 warnings, 1 hint unrelated to CBAM), the unit suite (261/261, of which CBAM's own share is 80/80 across `cbam-render.test.mjs` and `cbam-lines.test.mjs`), a production build, and the publication contract (80 checks across 15 routes, 0 violations)

— all pass, run today.

`npx playwright test tests/e2e/cbam-lines.spec.ts` — 19/19, covering add/remove/attest, both exports, the print-class cleanup paths, and one automated accessibility pass over a populated multi-line state.

`node scripts/cbam-sync-check.mjs` — the vendored engine still matches its committed manifest, and the shipped rule pack still matches upstream. Nothing in this branch touched the regulatory engine; only `cbam-app.ts` and `cbam-lines.ts`, which this repository owns outright, changed.

The formulas behind every figure above — SEE, SEFA, the Column A/B split, the production-route matrix — are in `cbam-engine-reference.pdf`. This brief does not repeat them.